

FURTHER_CATALYST_IMPLEMENTATION_STEPS

Version

v0.3.14: Catalyst pipeline planning + investigation of higher win-rate

Purpose

This version documents the clean pipeline for catalyst logic, adds the idea of restricting catalyst-selected symbols even further (e.g. top-N), and notes the need to investigate why higher win-rates ($\approx 63\%$ WR for Scenario D) were observed in past runs.

Current Baseline

- **Canonical outputs**: per-day results written to ``out/YYYYMMDD/SCENARIO/results_YYYY-MM-DD.csv``.
- **Range driver**: use ``scripts/run_range_safe.py`` (not ``run_range_and_summarize.py``) after clearing ``out/``.
- **Day driver**: ``scripts/run_day_simple.py`` works correctly after clearing ``out/``.
- **Analyzer**: ``scripts/analyze_base_only.py`` checks only canonical per-day files, avoiding ``out/auto`` artifacts.

Clearing the ``out/`` Directory

- The ``out/`` directory contains only **derived artifacts** (daily results, range summaries).
- It is **safe to delete** at any time, because all files are regenerated when you re-run backtests.
- We added a helper script:

```
```bash
python scripts/clear_out.py
```
```

This deletes ``out/`` completely and recreates it empty.

Why this matters

- After clearing ``out/``, we noticed that ``run_range_and_summarize.py`` produced summaries with **0 trades**.
- This happened because that script was summarizing without regenerating per-day result files first.
- To fix this, we ran ``run_range_safe.py``, which executes each day's backtest before building the summary. That gave correct totals again.

Best Practice

- **Run ``clear_out.py`` at the start of each session** to avoid stale artifacts affecting analyzers.
- Then use **``run_range_safe.py``** (not ``run_range_and_summarize.py``) to rebuild per-day files and summaries in one step.

- For single days, ``run_day_simple.py`` works fine immediately after ``clear_out.py``, since it regenerates everything needed.

Catalyst Pipeline (planned for v0.3.14)

1. ****Top gappers**** → generate universe list in descending gap% order.
2. ****Apply Ross-style filters**** → price 1–20, gap 10–40%, pre-market vol $\geq 30k$, etc.
3. ****Catalyst filter**** (Polygon News):
 - Keep symbols with real catalysts (FDA, trial, contract, upgrade, earnings beat).
 - Drop symbols with negatives (ATM/offering, dilution, reverse split, deficiency notices).
4. ****Optional top-N cap**** → restrict to top 3–5 catalyst-qualified symbols.
5. ****Write filtered universe file**** → ``data/catalyst/universe_YYYY-MM-DD.txt``.
6. ****Run backtests**** → feed filtered universe into ``run_day_simple.py`` or ``run_range_safe.py``.

Expected Effects

- ****Trade count****: reduced.
- ****Win rate****: likely higher.
- ****Expectancy****: improved, fewer outsized losers.
- ****Profitability****: potentially higher, capital concentrated on better setups.

Investigation: Higher Win Rate (~63%)

- Past Scenario D runs showed $\approx 63\%$ WR (e.g., August baseline). This was not due to a top-N cap but due to **restrictions on gappers** (price, gap, volume, MACD confirm, gate=5, TP/SL 2.0/2.5, etc.).
- Observed again in recent days: indicates conditions for higher WR still occur.
- Next steps:
 - Re-run D and E across August with base-only analyzer.
 - Compare with more recent days where higher WR was seen.
 - Identify if differences came from universes, scenario params, or sample size.

Next Actions

1. Implement Polygon-based catalyst enrichment (``scripts/enrich_universe_catalyst.py``).
2. Add optional ``--limit N`` flag for top-N restriction after catalyst filtering.
3. Re-test August dates and recent dates with catalyst+restricted universes.
4. Compare WR/PnL vs baseline runs.

Summary

****v0.3.14**** sets the roadmap: build catalysts → filter by Polygon news → optionally cap to top-N → run with safe drivers. Also, investigate why Scenario D sometimes achieves $\approx 63\%$ WR, and whether that effect repeats with catalysts.